Práctica UT1 · Gestor de procesos trabajadores

PSP · UT1. Introducción a la programación multiproceso

Introducción

En esta práctica vas a construir un pequeño sistema de procesos que colaboran: un proceso **gestor** (SumManager) lanza varios procesos **trabajadores** (SumWorker), les reparte órdenes, recoge sus respuestas, detecta cuándo alguno falla y los cierra de forma ordenada al terminar.

Es el mismo esquema que usan muchos sistemas reales: servidores que reparten peticiones entre varios procesos.

Parte 1. SumWorker, el proceso trabajador

SumWorker lee órdenes de su **entrada estándar**, una por línea, y escribe **una línea de respuesta** por cada orden en su **salida estándar**. No termina hasta recibir la orden EXIT (o CRASH).

Órdenes que debe admitir

Orden	Respuesta	Ejemplo
SUM a b	La suma de a y b	SUM 3 4 → 7
MUL a b	El producto de a y b	MUL 6 7 → 42
FACT n	El factorial de n (0 ≤ n ≤ 20)	FACT 5 → 120
PRIMO n	SI si n es primo, NO en otro caso	PRIMO 97 → SI
PID	El PID del propio worker	PID → 48213
CRASH	No responde: termina inmediatamente con código de salida 3 (simula un fallo)	—
EXIT	Responde BYE y termina con código de salida 0	EXIT → BYE

Órdenes no válidas

Si la orden no existe, le faltan o le sobran datos, o los datos no son correctos (por ejemplo, SUM 3 hola o FACT -2), el worker:

1. responde ERROR en su salida estándar, para que el gestor no se quede esperando;
2. escribe el motivo en su **salida de error**, indicando su PID. Por ejemplo: [PID 48213] Orden no válida: SUM 3 hola.

> **Importante**
>
> El worker **siempre** responde exactamente una línea por cada orden (salvo CRASH). Si no lo hace, el gestor se quedará esperando una respuesta que nunca llega o leerá respuestas que no corresponden a su orden.
>
> Pruébalo solo antes de seguir: ejecútalo desde IntelliJ y escribe las órdenes en la consola.

Parte 2. SumManager, el proceso gestor

2.1. Lanzar los workers

SumManager recibe por args el número de workers que debe lanzar (si no se indica, lanza 2). Al arrancar, los lanza y muestra el PID de cada uno.

2.2. Registro de errores

La salida de error de **todos** los workers debe **añadirse** al fichero workers.log, sin borrar lo que hubiera de ejecuciones anteriores. El gestor no lee estos mensajes: los escribe directamente cada worker en el fichero.

2.3. Reparto de órdenes

El gestor pide órdenes al usuario por teclado y las reparte entre los workers **por turnos**: la primera orden al worker 1, la segunda al worker 2, y así sucesivamente, volviendo a empezar por el primero. Cada respuesta se muestra indicando el PID del worker que la ha dado.

2.4. Detección de caídas y recuperación

Si un worker termina inesperadamente (por ejemplo, tras recibir CRASH), el gestor debe:

1. detectarlo: al intentar leer su respuesta, readLine() devuelve null;
2. informar del PID del worker caído y de su código de salida;
3. lanzar un worker nuevo que ocupe su lugar;
4. contabilizar la caída para el resumen final.

La orden CRASH no se reenvía al worker nuevo (volvería a provocar la caída). Cualquier otra orden cuya respuesta se haya perdido por una caída sí debe reenviarse.

2.5. Cierre ordenado

Cuando el usuario escribe SALIR, el gestor:

1. envía EXIT a cada worker y lee su respuesta BYE;
2. espera a cada worker **como máximo 2 segundos**;
3. si alguno no ha terminado en ese tiempo, lo termina con destroyForcibly();
4. muestra el código de salida de cada worker.

2.6. Resumen final

Antes de terminar, el gestor muestra un resumen con:

- el número de órdenes atendidas por cada worker (identificado por su PID);
- el número de respuestas ERROR;
- el número de caídas y de workers relanzados;
- el tiempo total de ejecución.

Parte 3. Ampliación: modo lote

Si se pasa un fichero de órdenes como segundo argumento, el gestor funciona en **modo lote**:

```
java -cp bin PSP.SumManager 3 ordenes.txt
```

- lee las órdenes del fichero, una por línea, en lugar de pedirlas por teclado;
- las reparte entre los workers igual que en la parte 2;
- escribe cada resultado en el fichero resultados.txt, indicando la orden, el PID del worker y la respuesta;
- al terminar el fichero, hace el cierre ordenado y muestra el resumen.

Ejemplo de ordenes.txt:

```
SUM 10 20
MUL 7 8
PRIMO 101
FACT 10
SUM 1
PRIMO 1000
```